

Networking Simplified



Handwritten By:
Vaibhav Matkar

Network → computers connected to each others

Internet → Network of computer Networks.

How internet started?

• **ARPANET** → Advanced research project Agency Network

↳ 1st computer network → Tcp/Ip based (1969)

↳ more computers added & Formed internet

↳ WWW by Tim-berner-lee (1989)



System of interlinked web pages accessed over internet using HTTP

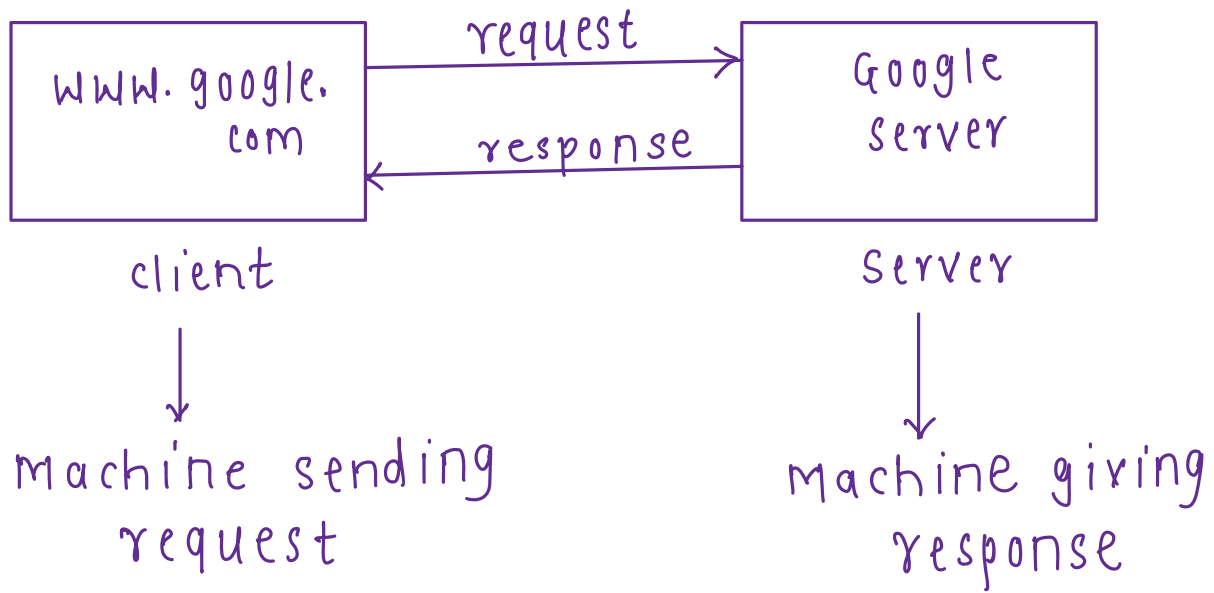
↳ problem of searching solved 1st with ARCHIE browser (FTP-based) in 1990

↳ Yahoo in 1994 → organised web pages manually

protocols → rules & regulations to send and receive data over internet.

↳ managed by internet society

client-server model



⇒ same machine can also become client as well as server

Important protocols

① TCP (transmission control protocol)

↓
communication protocol → ensures reliable, ordered and error free data transfer between computers over internet.
eg. tracking number / confirmation.

② UDP (user datagram protocol)

↓
communication protocol → fast but unreliable
used when speed is more important than accuracy.
eg. video streaming / VoIP calls.

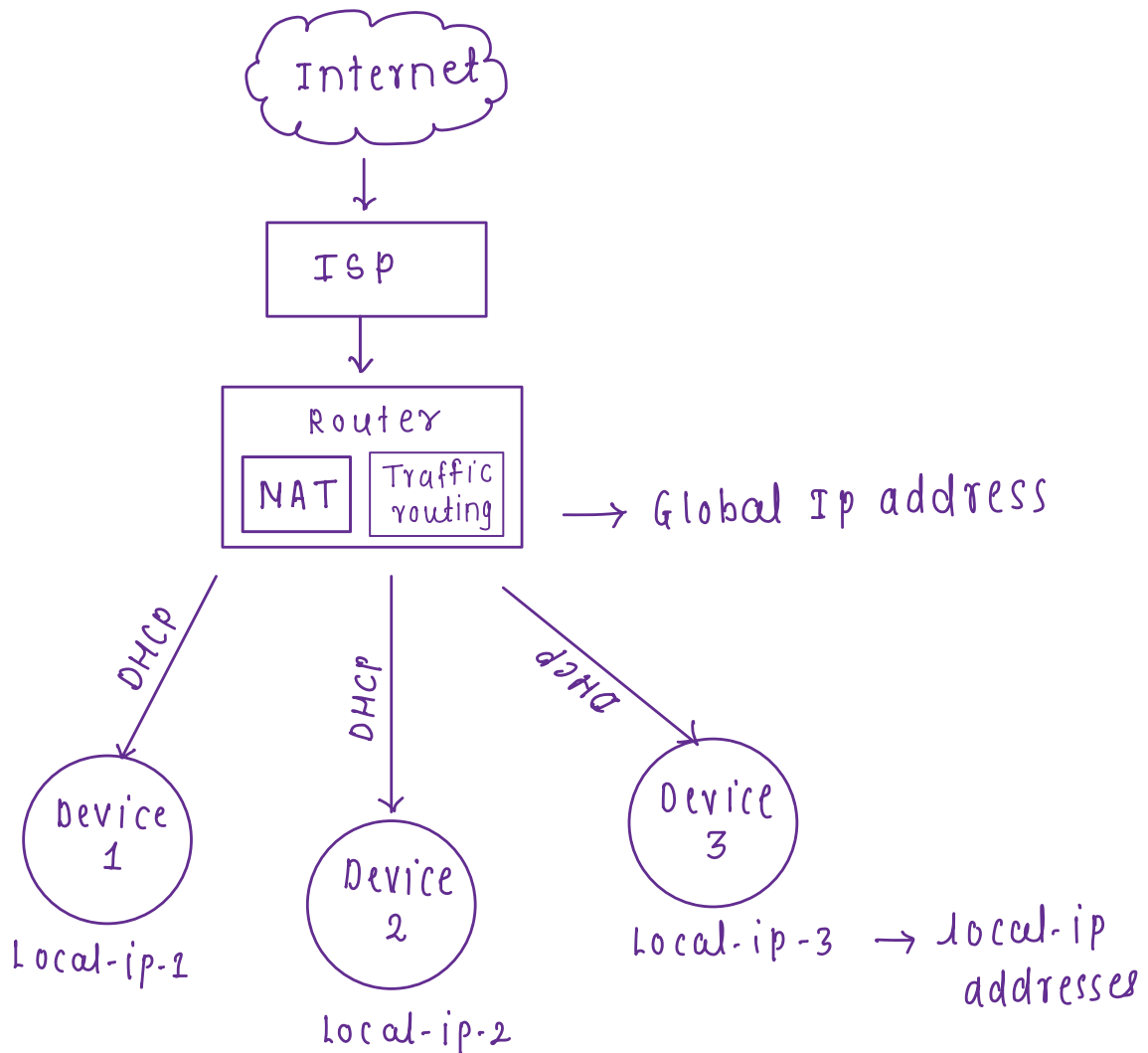
③ HTTP (HyperText Transfer protocol)

↓
protocol used for communication between web-browser and web-server. Tells how to request & response.

IP-address → gives identity to any machine/device

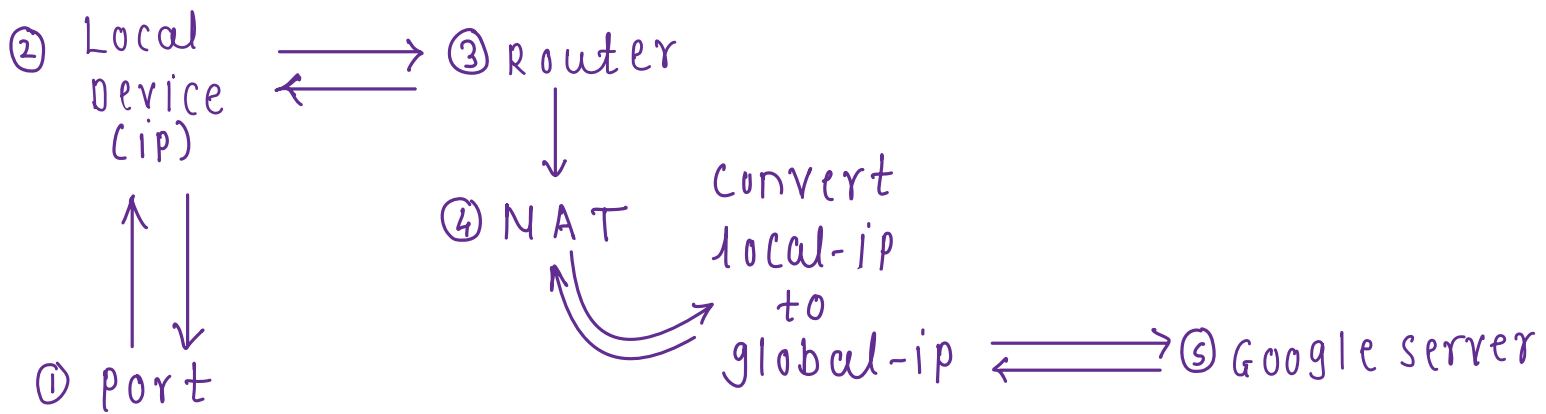
\$ curl ifconfig.me -s ⇒ gives ip of internet provider

How data is transferred?



- DHCP protocol gives local-ip address to connected devices

• request flow:



• **port** $\Rightarrow$ identifies application to send request/response.
i.e. ip $\rightarrow$ identify device but port $\rightarrow$ identify applⁿ
eg. mobile phone eg. Instagram.

$\hookrightarrow$ 16-bit number $\Rightarrow$ total ports $= 2^{16} \approx 65k$

$\hookrightarrow$ 0-1023 $\rightarrow$ reserved ports $\rightarrow$ system services

$\hookrightarrow$ 1024-49151 $\rightarrow$ registered ports $\rightarrow$ applications.

$\hookrightarrow$ 49152-65535 $\rightarrow$ Dynamic/ephemeral $\rightarrow$ temporary use

• **#Internet Speed** $\Rightarrow$ how fast data is uploaded/downloaded

• 1 kbps = 1000 bits/second

• 1 mbps = 10^6 bits/second

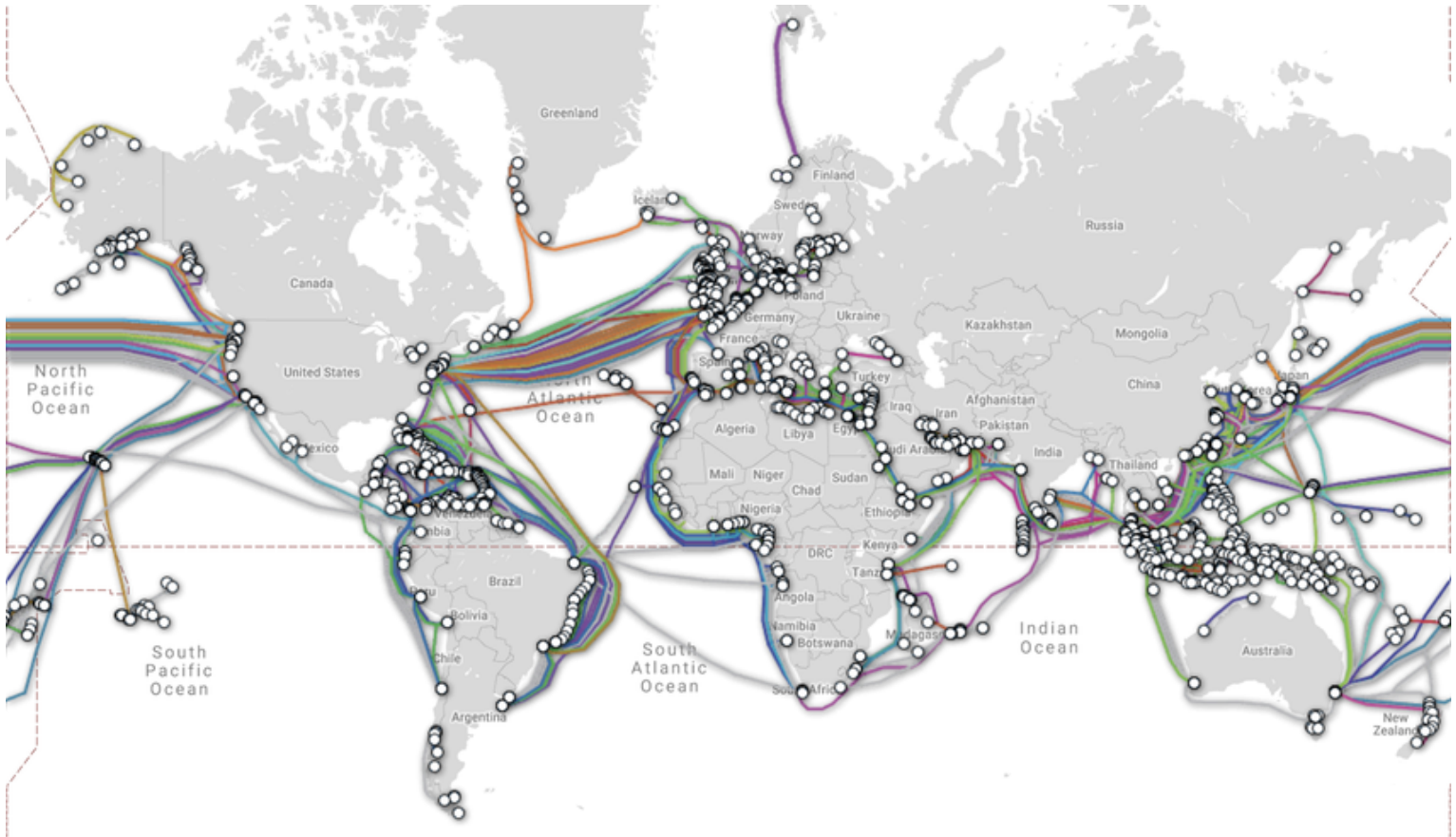
• 1 gbps = 10^9 bits/second

Ways of connecting devices

Guided way → connection path is defined. eg. devices connected using wires

Unguided way → path is wireless. eg. wifi / bluetooth / satellite

submarine-cables ⇒ Transfer data between countries using optical-fibre cables (about 98%)



Types of networks based on coverage area

- ① PAN (personal Area Network) → very small network
eg. Buds/ watch
- ② LAN (Local Area Network) → office/home specific
eg. wifi connectivity
- ③ MAN (Metropolitan Area Network) → city level
eg. cable TV network
- ④ WAN (Wide area Network) → covers large distance
eg. submarine wired network between countries.

• SONET (Synchronous optical network)

- ↳ faster communication / data transfer
- ↳ eg. submarine cables.

• Frame Relay

- ↳ packet switching WAN Technology that connects multiple LAN's using multiple logical connections (virtual circuits)
- ↳ supports flow of data on shared network using private connection.

modem → modulator + demodulator

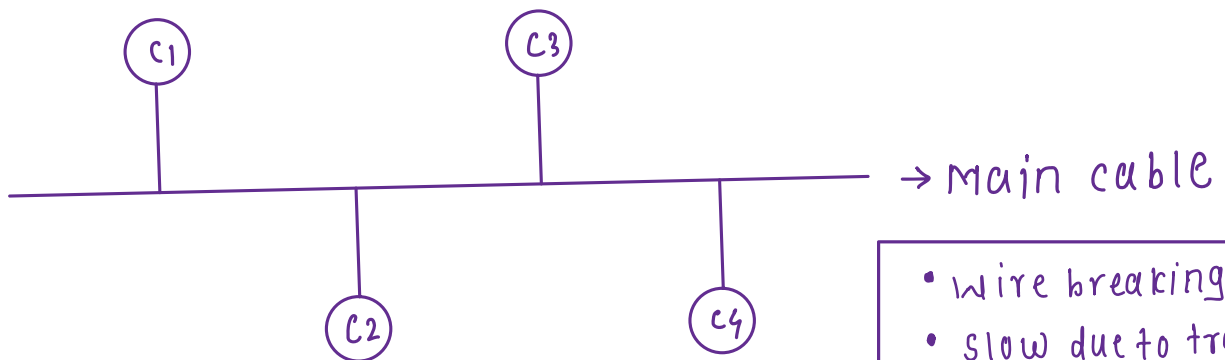
↳ converts one form of signal into another form

↳ eg. digital signal $\rightleftharpoons$ Analog signal

Router → distributes internet to multiple devices

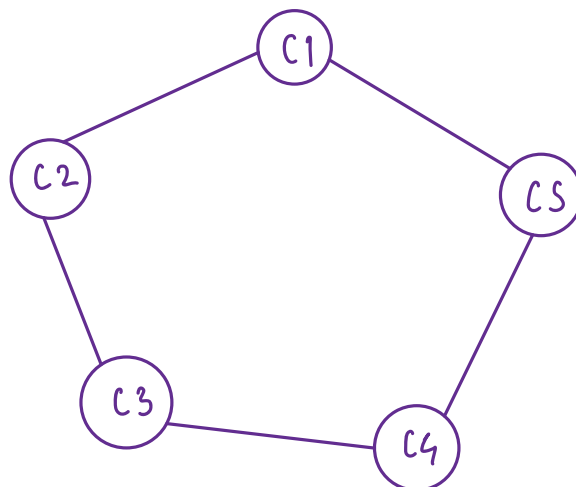
Topologies

① Bus topology



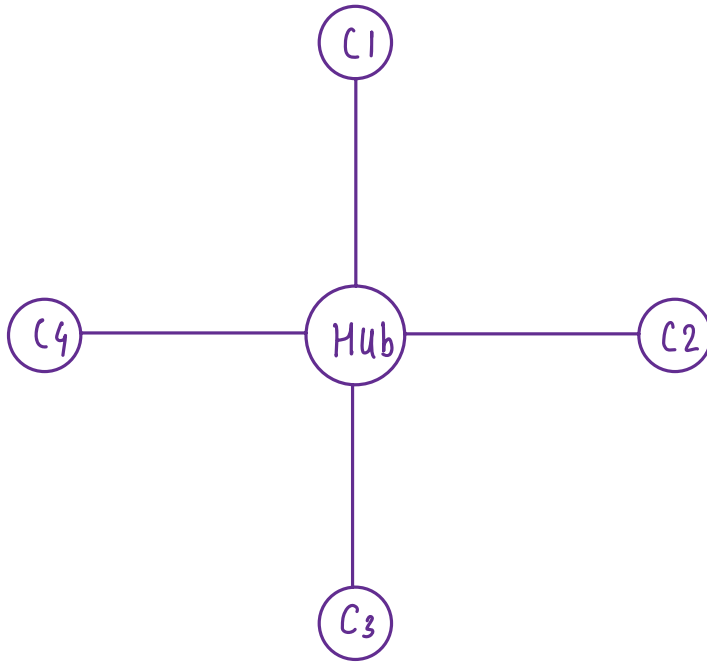
- wire breaking
- slow due to traffic

② Ring topology



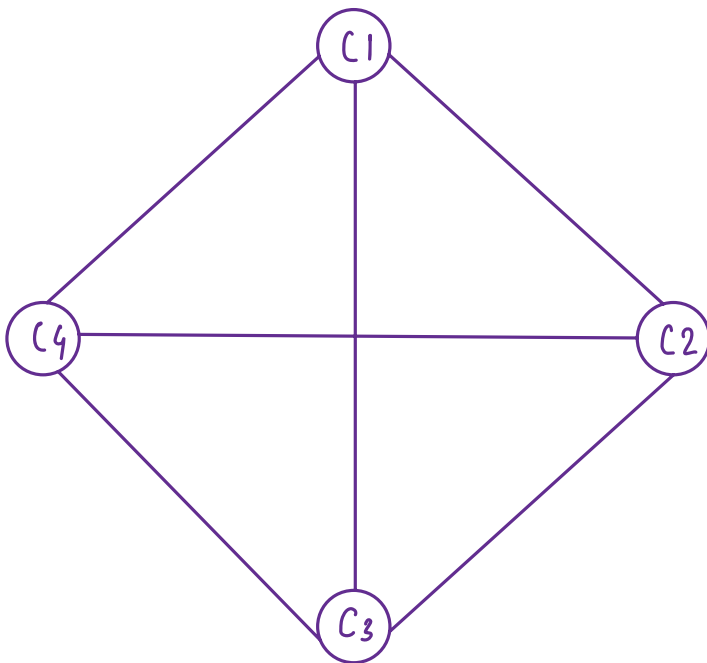
- 1 device fails
⇒ all network down

③ Star Topology



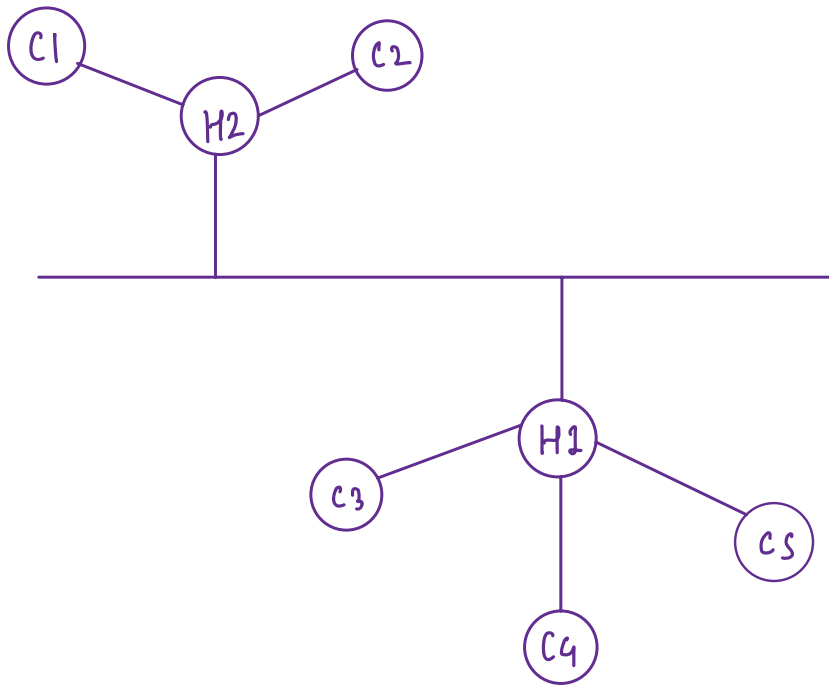
- Hub Down
⇒ Network Down

④ Mesh topology



- costly & complex

⑤ Tree topology (Bus + star)



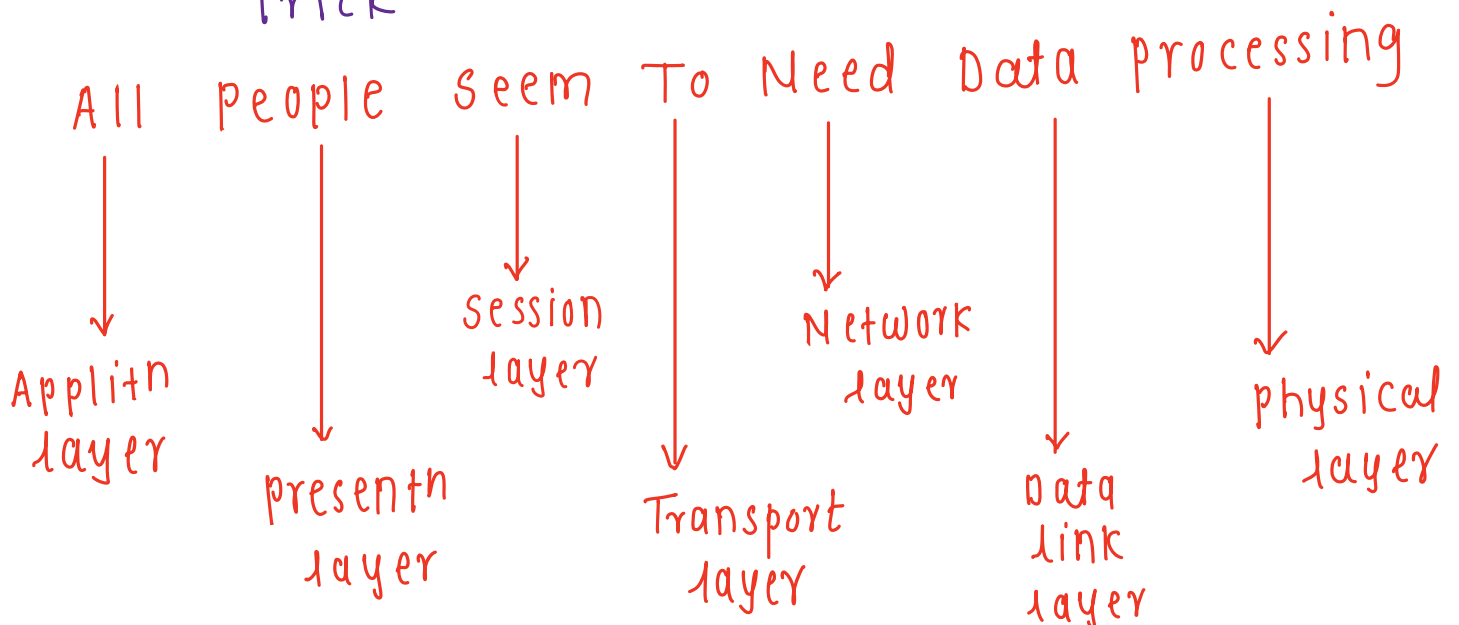
- Root device failure
=> outage

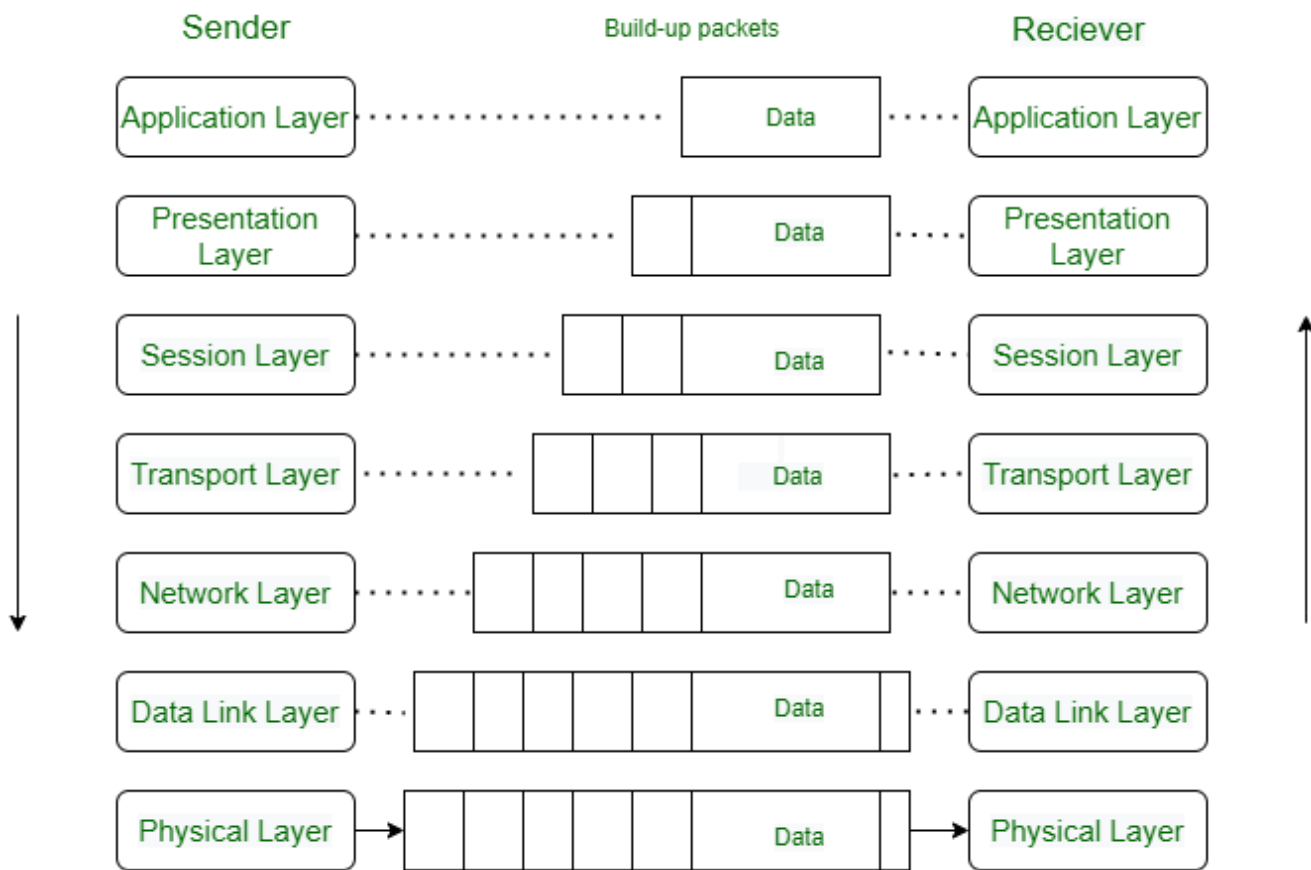
How internet works ? / OSI model

- OSI → open systems interconnection

↳ 7 layer model → describes how network communication happens.

↓
Trick





① Application layer

→ Involves software applications where user interacts

② presentation layer

→ recieves data from application layer & makes data machine readable + encrypt it + compress

③ session layer

→ responsible for managing sessions between client & server → Authentication & authorisation happens here.

④ Transport layer

↳ Here, data will be segmented (chunks)

↳ Each segment contains

- source port
- destination port + checksum
- sequence number → decides order of segments

↳ controls data flow (s → c)

↳ uses TCP & UDP protocols

⑤ Network Layer

↳ sends received data from one computer to another in different networks via routers

↳ Load balancing happens here

↳ Logical addressing → assign senders & requesters ip address to every segment & forms ip-packet.
→ to send each segment to correct destination.

⑥ Data link layer

↳ MAC addresses (physical) are assigned to the data packets which forms frames and those frames are sends bit-by bit to underlying hardware

↳ Logical Link Control (LLC) → provides error messages

⑦ physical layer

↳ Transfers raw bits between devices

↳ convert data into signals to travel through wires (modulation + demodulation)

↳ includes cables / connectors / plugs & receivers

Network devices

① **Repeater** → available at physical layer.
When signal becomes weak, it copy signal bit by bit & regenerate it at original strength.
It is 2 port device. They do not amplify signals.

② **Hub** → multipoint repeater eg. connector in star topology.

• Types of Hub

Active Hub

- own power supply
- clean, boost & relay signal
- Extend maximum distance between nodes

passive Hub

- Take power supply from active hubs
- only relay signal
- cannot used to extend distance between nodes

③ **Bridge** → operates at data link layer. It is repeater + filter content by reading source & destination MAC addresses. connects two LAN's in same network. 2 port device → Input & output

• Types of bridges

Transparent Bridges

- Invisible bridges
- block/forwards data according to MAC address

Translational Bridge

- connects 2 different networks
- shares data between 2 systems

Source routing Bridges

- designed for token ring
- sender decides route

④ **switch** → present at data link layer → multi-port device → checks data for correction before sending it to correct port.

⑤ **Router** → route data packets based on ip's. It belongs to network layer. connects WAN & LAN together & has dynamic routing table which makes decisions of routing data packets.

⑥ **Gateway** → connects two networks based on different networking models. Also called as protocol converters.

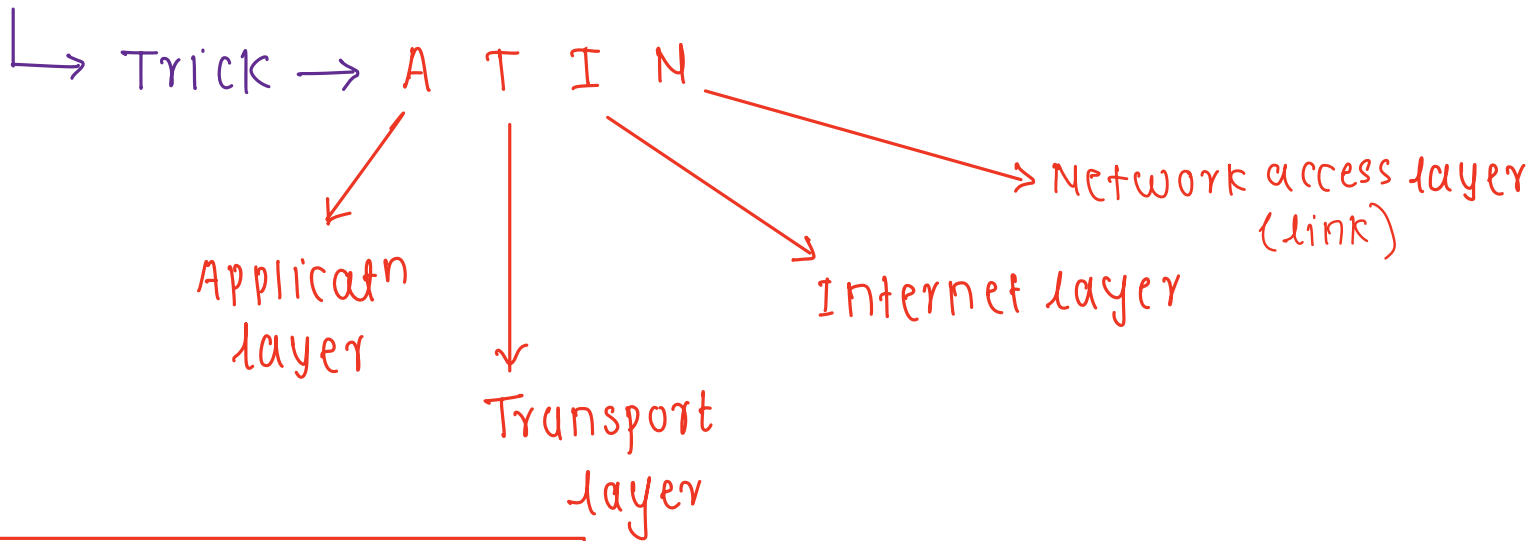
⑦ **Brouter** → (Bridge + Router) → @ D/Layer / Network layer

↓ ↓

filter local area network traffic Route packets

TCP-IP model

↳ practical - network model



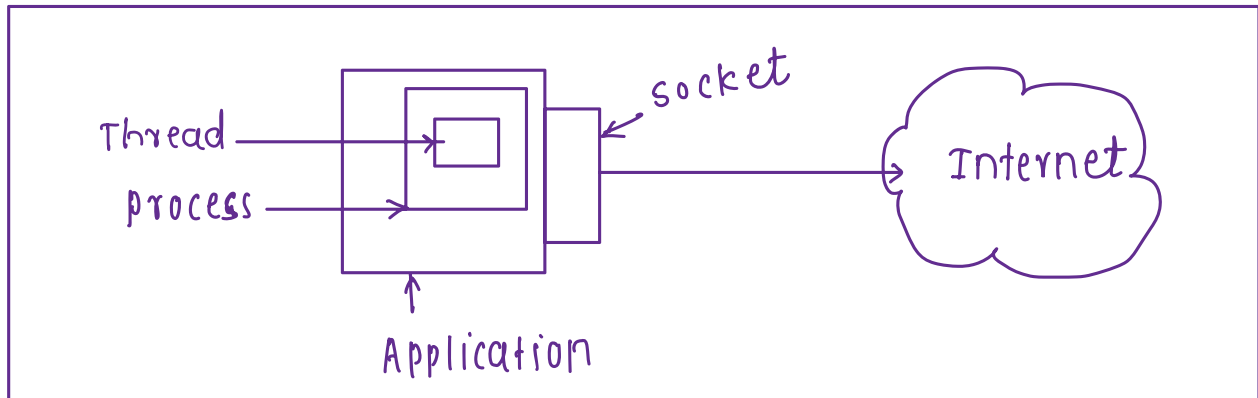
① Application layer

↳ contains applications where users interact
eg. whatsapp / browser

↳ protocols:

- ① HTTP → transferring data on web pages (req/resp)
- ② DHCP → allocates ip's to devices connected to network
- ③ FTP → file transfer protocol
- ④ SMTP → send email
- ⑤ POP3 & IMAP → download/receive email + sync email
- ⑥ SSH → connecting securely to shell (CLI)
- ⑦ telnet → similar to ssh + unsecure
- ⑧ VNC → virtual Network computing → Remote Desktop protocol. (GUI)
- ⑨ UDP → fast & unreliable data transmission

- **program** → Application → eg. Whatsapp
- **process** → feature → eg. sending a message
- **thread** → sub-feature → eg. displaying keyboard
- **socket** → made up of (ip address + port No) i.e. used as an endpoint for sent/receive data



HTTP Methods

- ① GET → requesting something from server
- ② POST → Giving something to server eg. ID & password
- ③ PUT → replaces already existing data
- ④ DELETE → delete data from server.

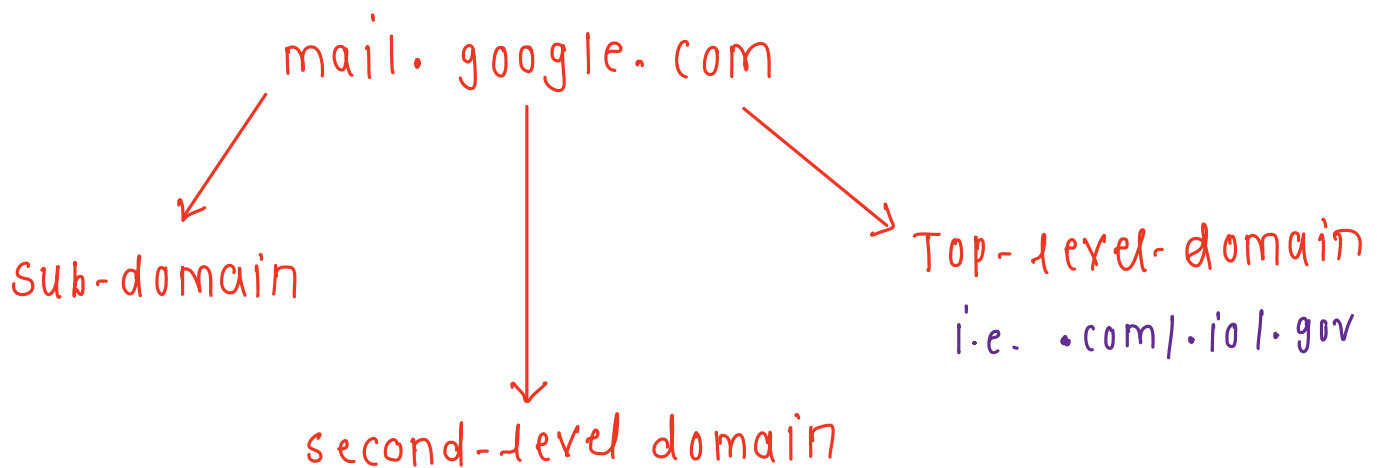
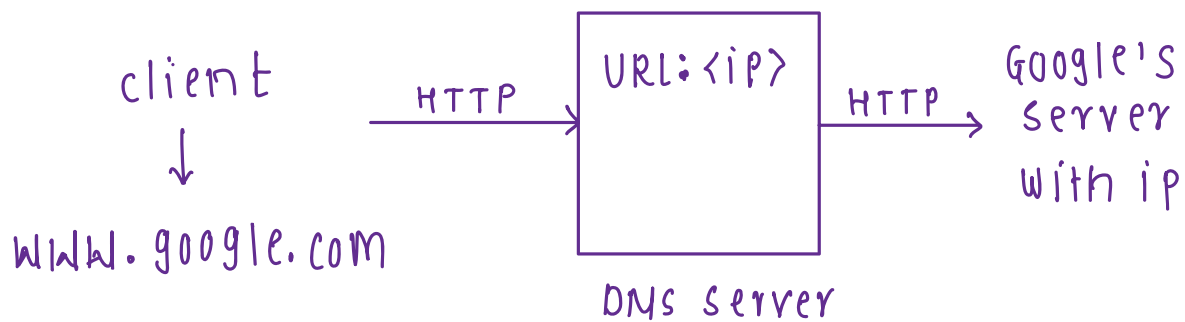
HTTP status codes

- ① 1xx → informational → eg. 100. continue
- ② 2xx → Success related → eg. 200 → all OK.
- ③ 3xx → Redirection related → eg. 301 moved permanently
- ④ 4xx → client errors → eg. 404 - resource not found.
- ⑤ 5xx → server errors eg. 500 - internal server error

#cookies → small peice of data (string) stored in browser to maintain user sessions & preferences

#3rd party cookies → cookie of a any 3rd party website (contains ads/analytics) kept in a website we have opened.

#Domain Name system (DNS)



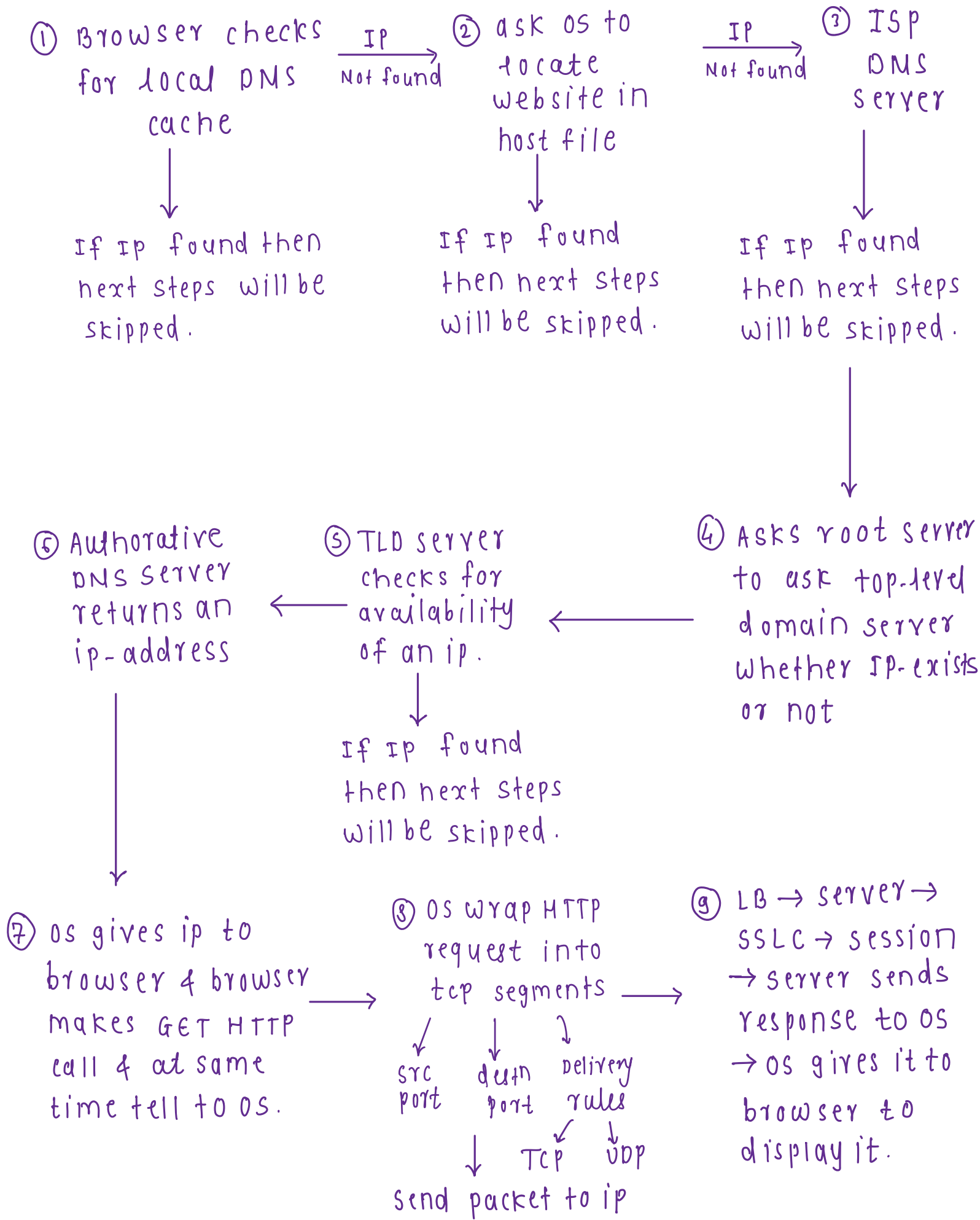
Root DNS servers

↳ Top-level DNS servers which guides domain name-query to found Top Level-Domain server.

↳ req. → Root DNS server → TLD server → Authorative DNS server

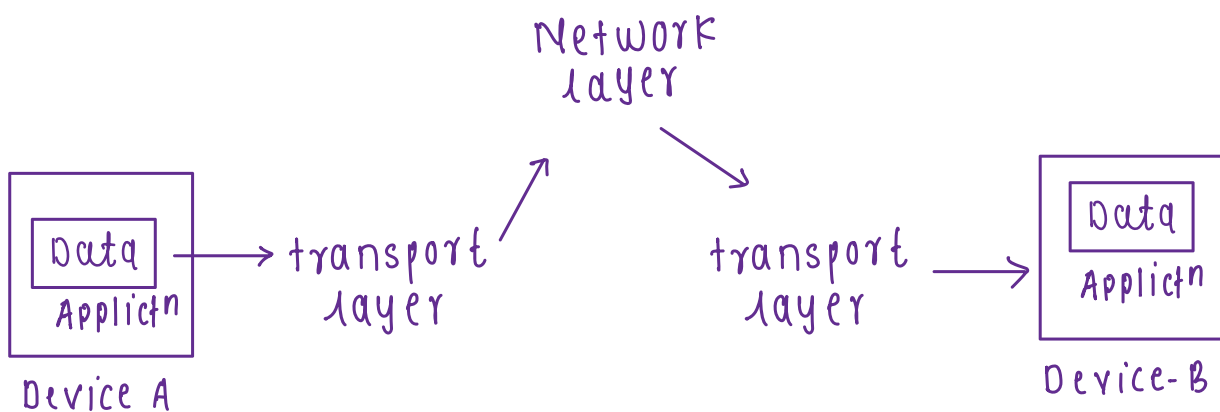
eg. ICANN eg. .in → NIXI eg. Godaddy.

• What happens when I click www.google.com?



② Transport Layer

- Network layer → send data from one user to other via network
- transport layer → sends correct data to an correct application on device.



transport layer multiplex → sending multiple data streams by combining it.

- Example ⇒ sending (whatsapp + email + snap data) on single ip

transport layer demultiplex → distributing data come from one network to correct application

- Example ⇒ see correct port & distribute data to correct applications.

congestion control → Tcp mechanism to regulate data transmission to prevent network overload.

① slow start → ② congestion avoidance → ③ resends packets if lost → ④ slow speed.
↓
exponential ↑

checksum in tcp

↳ Used to detect errors in transmitted data to ensure data integrity.

↳ It is added by sender in tcp header, this value should be same when checked by receiver.

Timers in tcp

↳ used to check whether data is transmitting in correct time or not.

① Transmission timer (RTO)

- sends packet → If NO ACK → resend packet

② persistence timer

- receiver is not ready but keeps checking availability.

③ keepalive timer

- checks if connection is alive or not

④ Time-wait (2MSL) timer

- wait some time if connection closes, to avoid duplicate packet sending.

Transport layer protocols

① UDP → (User Datagram protocol)

↳ data may deliver/not but faster

↳ data may change

↳ data may not in order.

↳ connectionless protocol

↳

source port (2 byte)	destination port (2 byte)	length of data stream (2 byte)	checksum (2 byte)
Actual data to transmit ($2^{16} - 8$ bytes)			

↳ UDP data packet (2^{16} bytes)

↳ use-cases

- video conferencing
- DNS
- Gaming

↳ `sudo tcpdump -c <packet-count>`

↳ capture 'n' packets from CLI

② Tcp (Transmission control protocol)

↳ Transport layer protocol

↳ raw data from Applctn layer $\rightsquigarrow$ tcp $\rightarrow$ fragments data into chunks & adds headers

+

checks data recieved from network layer for potential errors at recievers end side

↳ reliable data sending

↳ congestion management

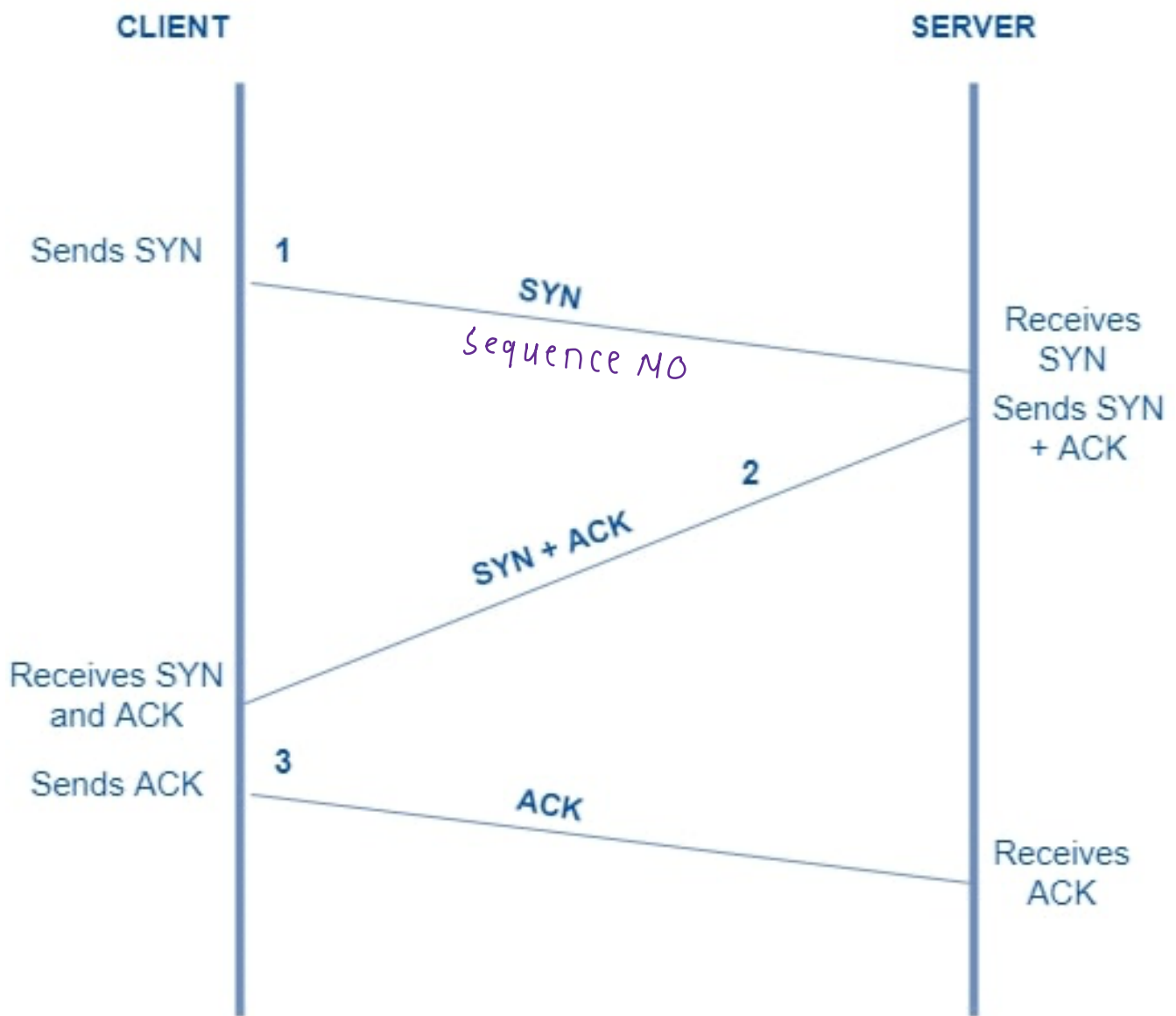
↳ maintains order of data $\rightarrow$ sequence Numbers

↳ Tcp $\begin{cases} \rightarrow \text{pdfs} \\ \rightarrow \text{HTTP} \\ \rightarrow \text{Emails} \end{cases}$

↳ connection oriented (1st connection $\rightarrow$ Transfer)

3-way-Handshake

↳ method to establish connection between server & client.

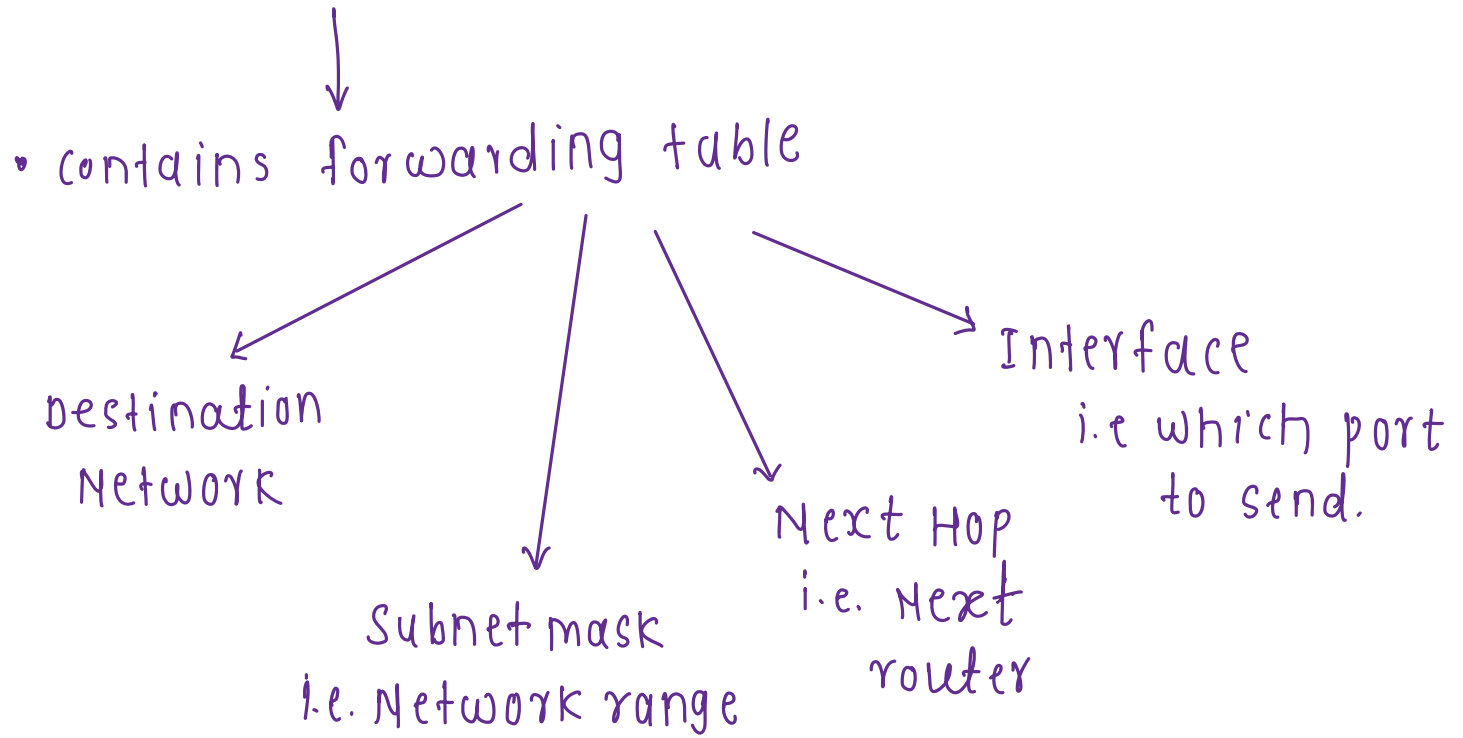


- $ACK\ NO = prev.\ SYN\ NO + 1$

③ Network Layer

- Transport layer $\rightarrow$ data in forms of segments
- Network layer $\rightarrow$ data in forms packets
- Data link layer $\rightarrow$ data in forms of frames.

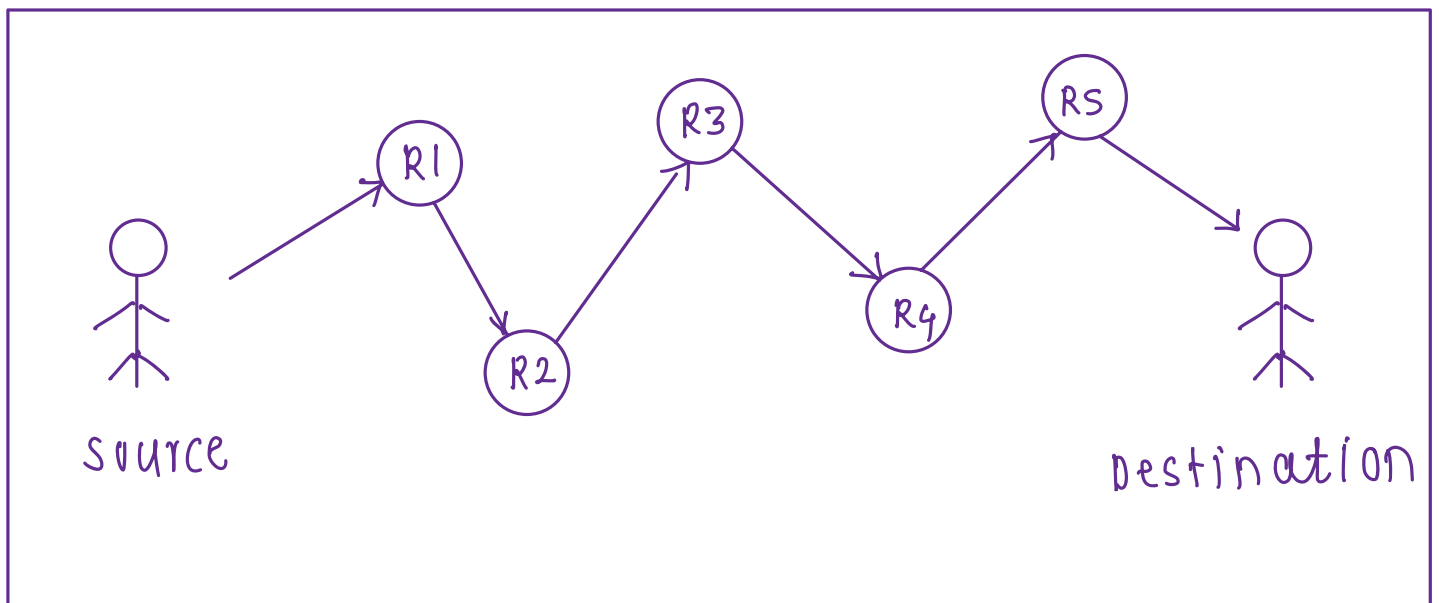
- Here, routers comes into the picture.



- contains routing table

↓

contains information about available routes to different networks + determines best path



• **control plane** → creates routing tables

static routing

- manually configuring routes

dynamic routing

- routes are defined automatically

• **Network layer protocols**

• **IP (internet protocol)**

① IPv4 → 32-bits → 4 words/octets

② IPv6 → 128 bits → 8 words/octets

• **structure of an ip-address**

192.168.1.10

Network
address

↓
Identify
network

Host
Address

↓
Identify device
inside network

• **subnet masking**

↳ Tells router where network address ends
and host address starts

eg. for ip → 192.168.1.10

subnet mask → 255.255.255.0

- **CIDR** → method of writing IP & subnet mask together

eg. 192.168.1.0/24

↑ 1st 24 bits ⇒ Network
remaining bits ⇒ Device

- Each bit in network part = 1
- Each bit in host part = 0

$\underbrace{11111111}_{255} . \underbrace{11111111}_{255} . \underbrace{11111111}_{255} . \underbrace{00000000}_{000}$
 255.255.255.000

classes of an ip-address

① class-A

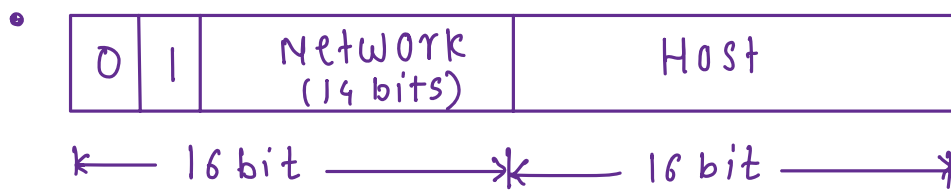
- Network having large number of hosts



- 1st bit = 0
- default subnet mask → 255.0.0.0 / ip/8
- Host address possible = $2^{24} - 2$ (Network + broadcast addr)
- ip-address range ⇒ 0.0.0.0 → 127.255.255.255

② class-B

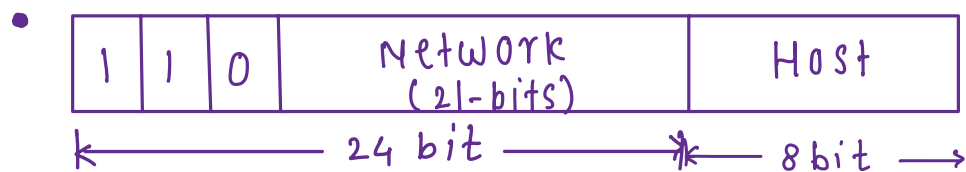
- assigned to medium to large sized networks



- 1st two bits $\Rightarrow 01$
- default subnet mask $\rightarrow 255.255.0.0 / ip/16$
- Total No of Networks $= 2^{14} = 16,384$ networks
- Total No of Hosts $= 2^{16} - 2 = 65,534$ hosts
- Ip-range $\Rightarrow 128.0.0.0 - 191.255.255.255$

③ class-C

- assigned to all small sized networks



- 1st three bits $\Rightarrow 110$
- default subnet mask $\Rightarrow 255.255.255.0 / ip/24$
- Total No of Networks $= 2^{21} = 20,97,152$ networks
- Total No of Hosts $= 2^8 - 2 = 254$ hosts
- ip-range $\Rightarrow 192.0.0.0 - 223.255.255.255$

④ class-D

- reserved for multicast communication.
eg. live streaming/online class (1 → n)



← 4 bit → 28 bit →

- first 4 bits $\Rightarrow$ 1110 + 28 bits $\Rightarrow$ multicast group address where hosts can join.
- No network ID/ host ID division
- No subnet masking defined.
- IP-range $\Rightarrow$ 224.0.0.0 - 239.255.255.255

⑤ class-E

- for research & experimental purposes.
- 1st 4 bits $\Rightarrow$ 1111
- No network ID/ host ID division
- No subnet masking defined.
- IP-range $\Rightarrow$ 240.0.0.0 - 255.255.255.255

special
ip
addresses

① Link local addresses $\rightarrow$ used when device cannot get ip from DHCP server.

• 169.254.0.0 $\rightarrow$ 169.254.255.255

② loopback addresses $\rightarrow$ test network locally $\rightarrow$ 127.0.0.0 $\rightarrow$ 127.255.255.255

③ current network $\rightarrow$ initially used b/c device has valid ip $\rightarrow$ 0.0.0.0 $\rightarrow$ 0.255.255.255

packet structure → 20 bytes ↗ Header
↘ data

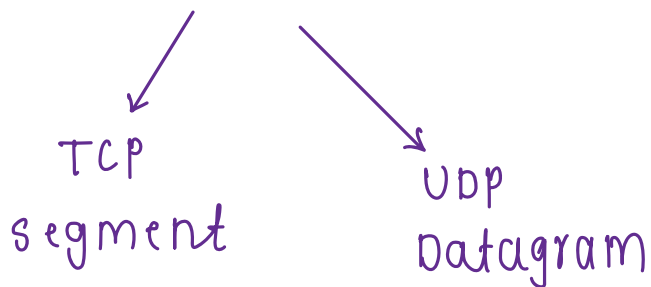
① Head → contains routing & delivery info

Important IP Header Fields and Their Functions

Field	Purpose
Version	Specifies IPv4 or IPv6
Header Length (IHL)	Size of the IP header
Total Length	Total size of the packet (header + data)
Identification	Identifies fragments of a packet
Flags	Controls fragmentation
Fragment Offset	Position of a fragment
TTL (Time To Live)	Limits packet lifetime to avoid loops
Protocol	Identifies next-layer protocol (TCP=6, UDP=17)
Header Checksum	Error detection for the header
Source IP Address	Sender's IP address
Destination IP Address	Receiver's IP address

② data / payload

↳ contains data from transport layer



↳ This is information application wants to send.

IPv6 → 128 bit address

$$2^{32 \times 4} = 2^{128} \downarrow = 3.4 \times 10^{38} \text{ uniq ip addresses}$$

pros

- ① Large address space
- ② No need of NAT
- ③ faster & efficient routing
- ④ Build in security

cons

- ① complex address format
- ② Backward compatibility issues
- ③ migration cost
- ④ Isp adoption issues

• structure of ipv6

• 128 bits → 8 groups → Each group → 16 bits

↓
Hexadecimal numbers
(0-9, a-f)

• | Global Routing Index | subnet ID | Interface ID

eg. 2001:db8:abcd:0012:0000:0000:0000:0001

• IPv6 packet header → 40 Bytes

middleboxes

↳ networking device → sits between sender & receiver

↳ functions → traffic →
 ↗ Inspect
 → Modify
 ↘ control/mgmt

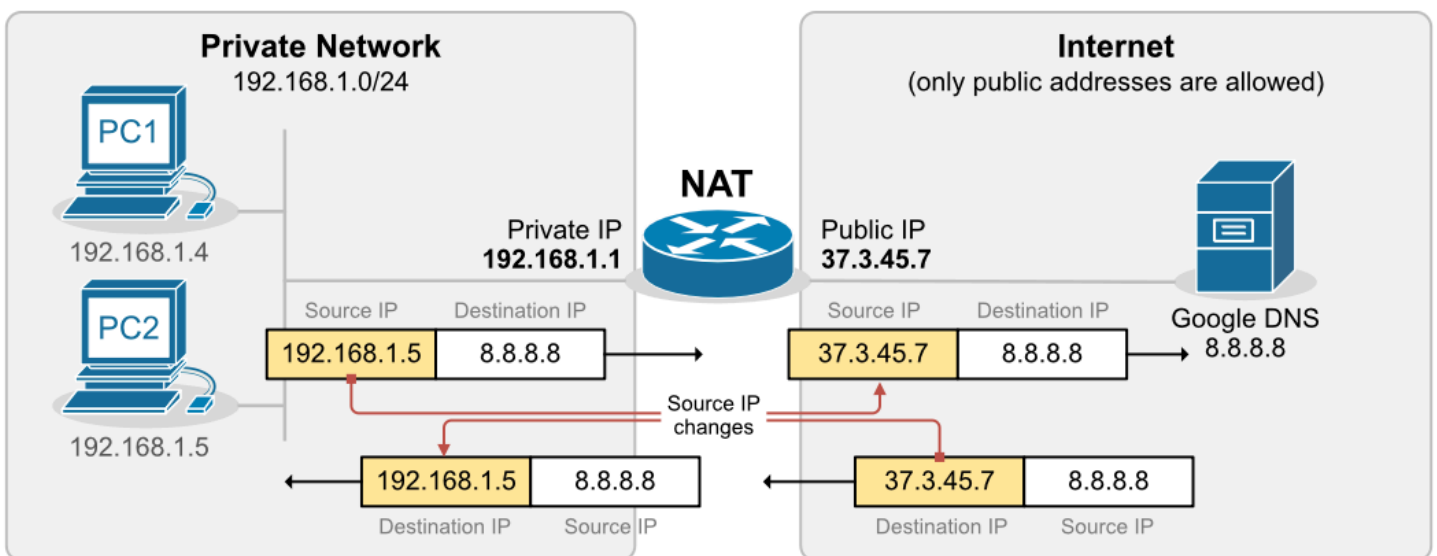
① Firewall → filters ip-addresses based on rules.

↙
stateful

↘
stateless

② NAT (Network Address translation)

→ convert private ip to public ip → serves it



④ Data link layer

↳ Node to node delivery of data in same network

↳ DHCP server $\Rightarrow$ pool of ip addresses $\Rightarrow$ Assign ip-addresses to devices connected to router.

• sub-layers

① Logical link control

↳ deals with multiplexing / provides error messages (checksum) & acknowledgements.

② media Access control

↳ receives data from network layer in the form of packets $\rightarrow$ divide packets into frames and send those frames bit by bit to underlying hardware. Here, MAC addresses ensures that data is sent to correct device.

• protocols in data link layer

① Ethernet $\rightarrow$ LAN protocol

② Wifi $\rightarrow$ Wifi LAN protocol

③ ARP $\rightarrow$ Address Resolution protocol $\rightarrow$ map ip $\rightarrow$ MAC

④ PPP $\rightarrow$ point to point protocol $\rightarrow$ direct connections

⑤ HDLC $\rightarrow$ Bit oriented protocol $\rightarrow$ WAN connections

Load Balancing → distributing incoming traffic to multiple servers. ⇒ High availability

• Types of load Balancers

Layer 4 LB @ transport layer

- src ip + port → destination ip + port @ traffic base
- tcp & udp protocols
- eg. Network Load Balancer (NLB)

Layer 7 LB @ Application layer

- Request based routing
- HTTP/HTTPS protocols
- eg. Application load balancer (ALB)

content delivery network (CDN)

↳ Globally distributed network of servers that helps to deliver web content with HA & low latency.

↳ cache content closer to users → reducing load on original servers

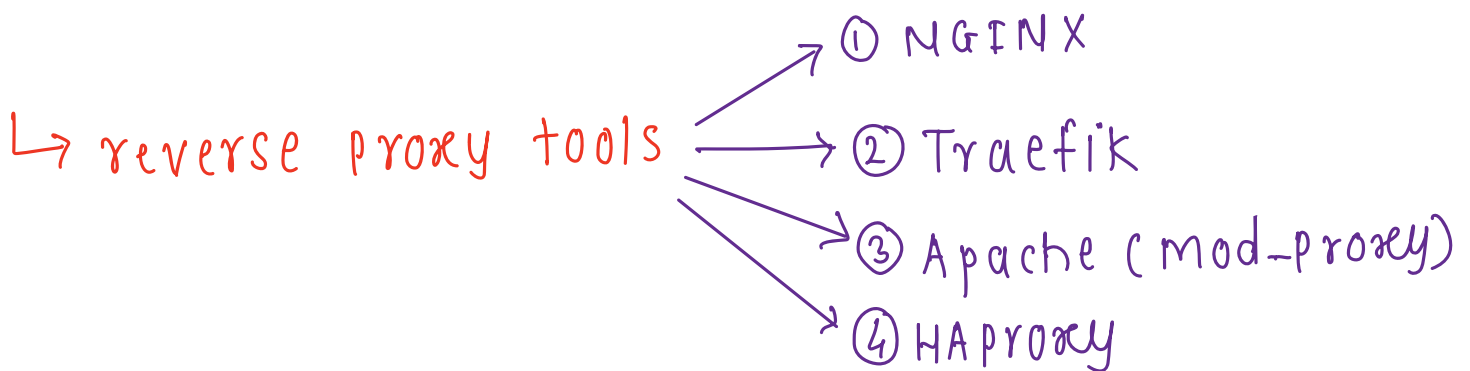
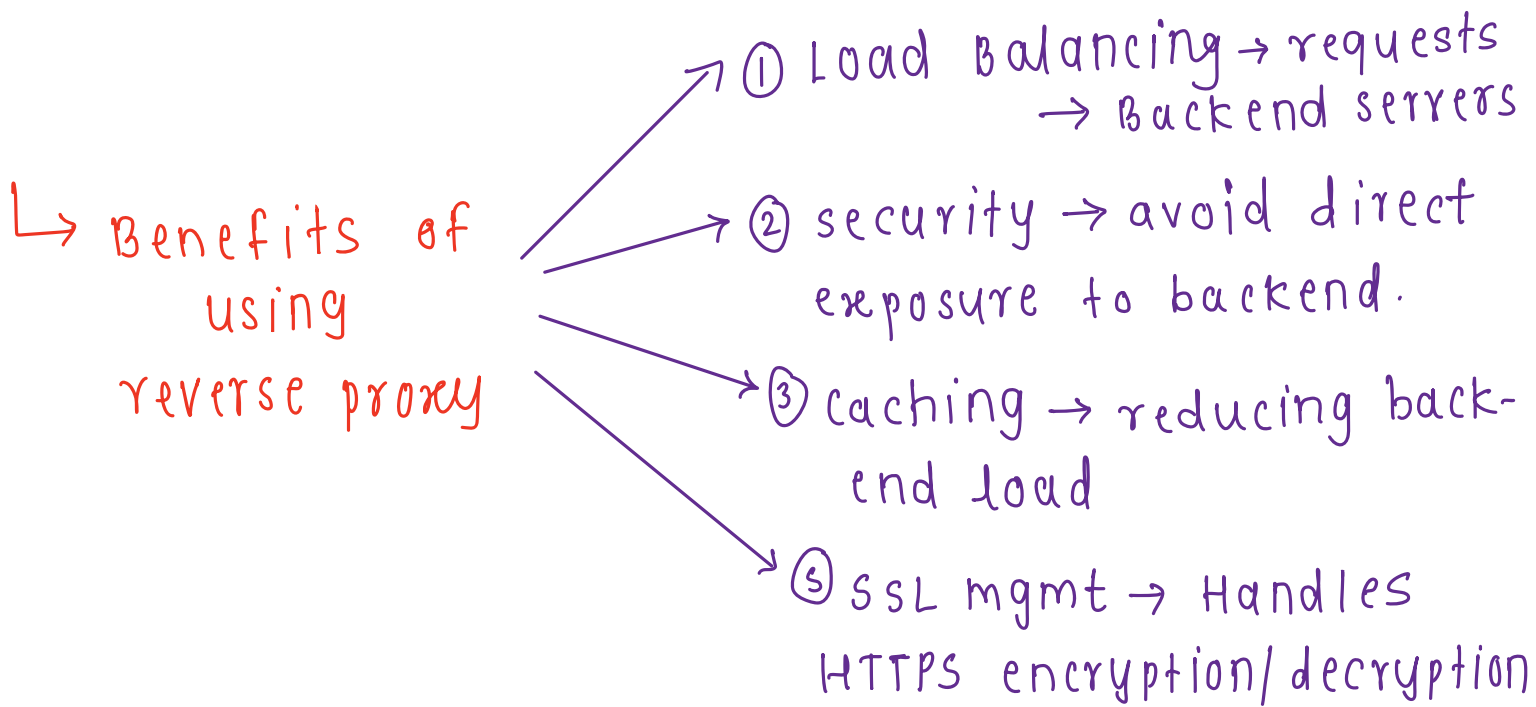
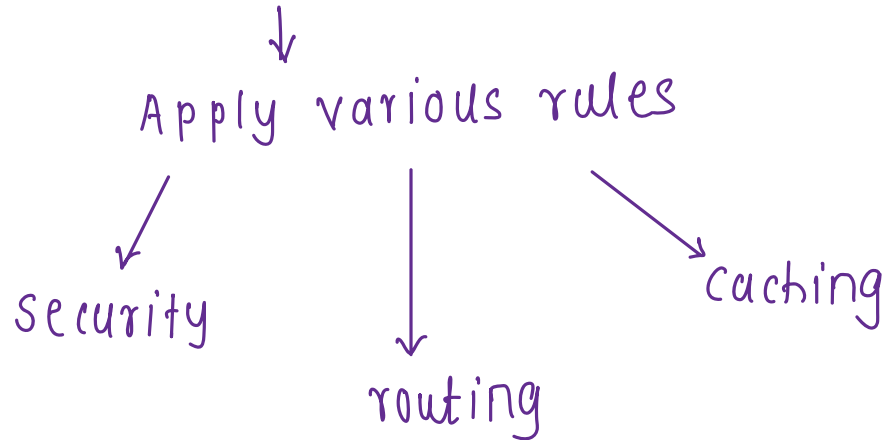
↳ popular CDN providers

- cloudflare
- Akamai
- AWS cloudfront
- Google cloud CDN.

Reverse proxies

↳ server that sits between client requests and backend servers.

i.e client → reverse proxy → backend



- Nginx reverse proxy file location.

/etc/nginx/conf.d/reverse-proxy.conf.

OR

/etc/nginx/sites-available/

- structure of reverse-proxy.conf.

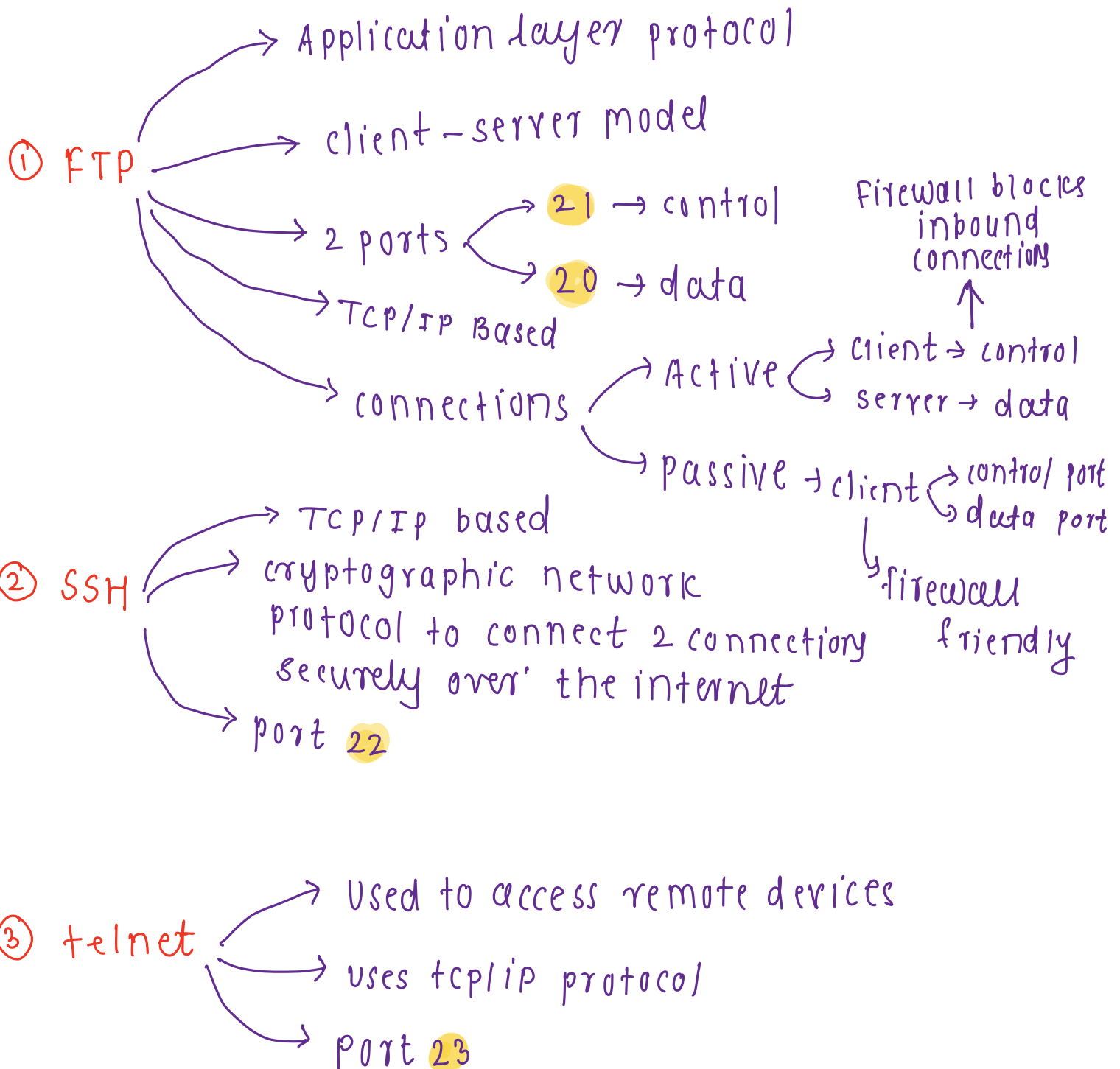
```
server {  
    listen 80;  
    server_name example.com;  
  
    location / {  
        proxy_pass http://192.168.1.100:8080;  
  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Networking tools for devops

- ① ping → Host reachability
- ② traceroute/ traceroute6 → show path & delay of packets across internet
- ③ netstat → displays active connections, routing tables, interface statistics, masquerade connections & multicast memberships
- ④ Nmap → Search hosts & services in computer network
- ⑤ tcpdump → capture & analyze internet traffic
- ⑥ ipconfig → To see configuration of network interfaces.
- ⑦ dig → To query depth DNS related problems
 - ↳ Advanced DNS queries
 - ↳ Domain Information Groper
- ⑧ nslookup → To query DNS to get domain related information
 - ↳ simpler DNS queries

network protocols

Rules that governs how data is transmitted & received between devices in a network.



④ SMTP

- Application layer protocol
- send receive emails
- port 25

⑥ DNS

- Used for domain name resolution
- Application layer protocol
- port - 53
- TCP/UDP based

⑦ DHCP

- Application layer protocol
- Assign ip addresses & other network configurations to devices automatically.
- Server → 67 & client 68

⑧ HTTP

- communication between client server on www
- uses TCP - 80 port

⑨ POP

- retrieve emails from email server
- Works in offline mode also
- Uses TCP - 110 port

⑩ NTP

- keeps computer clock time in synch in network
- uses hierarchical system at top with GPS/Atomic clock
- Uses UDP-123 port

⑪ snmp

- Manage & monitor network devices
- Uses UDP - 161/162

⑫ HTTPS

- secure version of http
- Uses Secure socket layer (SSL) & Transport layer security (TLS) protocol for communication (Encryption)
- Uses TCP 443 port